Treehouse SpectraWrappedtAVAX Audit Report

Oct 23, 2025

Table of Contents

Summary

This report has been prepared for Treehouse smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

Overview

Project Summary

Project Name	**Treehouse**
Codebase	**https://github.com/treehouse-gaia/tAVAX-protocol**
Commit	**ed99d50b0462ddf016d3d10702e6655424036e62**
Language	**Solidity**

Audit Summary

Delivery Date	**Oct 23, 2025**
Audit Methodology	**Static Analysis, Manual Review**
Total Isssues	**3**

[WP-M1] Using `ERC1967Proxy` directly, instead of the common `TransparentUpgradeableProxy` or UUPS + `ERC1967Proxy`, will result in a contract that cannot be upgraded.

Medium

Issue Description

https://docs.openzeppelin.com/contracts/5.x/api/proxy#transparent-vs-uups-proxies

> UUPS proxies are implemented using an ERC1967Proxy. Note that ***this proxy is ==not== by itself upgradeable***. It is the role of the implementation to include, alongside the contract's logic, all the code necessary to update the implementation's address that is stored at a specific slot in the proxy's storage space. This is where the UUPSUpgradeable contract comes in. Inheriting from it (and overriding the _authorizeUpgrade function with the relevant access control mechanism) will turn your contract into a UUPS compliant implementation.

`SpectraWrappedtAVAX` is not `UUPSUpgradeable`, and using `ERC1967Proxy` with `SpectraWrappedtAVAX` does not provide any upgrade functionality.

It's unclear what the purpose of using `ERC1967Proxy` is in this case. If upgrade functionality is not required, it would be better to not use a proxy at all (like a regular non-upgradeable contract, directly using `SpectraWrappedtAVAX`'s storage) to avoid the unnecessary gas costs from proxy DELEGATECALL operations.

https://github.com/treehouse-gaia/spectra-wrapped-tavax/blob/a4526cab672ad5ea42ca7174603f8a27fad0c1ff/script/DeploySpectraWrappedtAVAX.s.sol

```
@@ 1,5 @@
6  import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
7
8  /**
9   * @title DeploySpectraWrappedtAVAX
10  * @notice Deployment script for SpectraWrappedtAVAX using ERC1967 proxy pattern
```

```
@@ 11,20 @@
 */
contract DeploySpectraWrappedtAVAX is Script {
@@ 23,53 @@

    function run() external {
@@ 56,69 @@

        // Step 1: Deploy implementation contract
@@ 72,74 @@

        // Step 2: Prepare initialization data
        console.log("Step 2: Preparing initialization data...");
        bytes memory initData = abi.encodeCall(
            SpectraWrappedtAVAX.initialize,
            (
                wAVAX,
                sAVAX,
                tAVAX,
                treehouseRouter,
                initialAuthority
            )
        );

        // Step 3: Deploy proxy with initialization
        console.log("Step 3: Deploying proxy and initializing...");
        proxy = new ERC1967Proxy(address(implementation), initData);
        console.log("Proxy deployed at:", address(proxy));

@@ 94,126 @@
    }

@@ 129,148 @@
}
```

Status

✓ Fixed

[WP-L2] The current implementation heavily depends on the behavior of TreehouseRouter

Low

Issue Description

```
129  function _deposit(
130      address caller,
131      address receiver,
132      uint256 assets,
133      uint256 shares
134  ) internal override(ERC4626Upgradeable) {
135      SafeERC20.safeTransferFrom(IERC20(asset()), caller, address(this), assets);
136      _wrapperDeposit(assets);
137      _mint(receiver, shares);
138      emit Deposit(caller, receiver, assets, shares);
139  }
140
141  /*////////////////////////////////////////////////////////////////
142                        INTERNALS
143  ////////////////////////////////////////////////////////////////*/
144
145  /// @dev Internal function to mint tAVAX shares by first depositing in the sAVAX
     vault.
146  function _wrapperDeposit(uint256 amount) internal {
147      if (amount != 0) {
148          // Treehouse router handles the deposit of wAVAX into sAVAX and then into
     tAVAX. Returns tAVAX to this contract.
149          ITreehouseRouter(treehouseRouter).deposit(wAVAX, amount);
150      }
151  }
```

Consider disabling mint (do not support depositing wAVAX by specifying shares), adding `nonReentrant` modifier to `deposit()`, `wrap()`, `unwrap()` etc., and modifying `_deposit()` to:

```
function _deposit(
    address caller,
```

```
    address receiver,
    uint256 assets,
    uint256 shares
) internal override(ERC4626Upgradeable) {
    SafeERC20.safeTransferFrom(IERC20(asset()), caller, address(this), assets);
-    _wrapperDeposit(assets);
+    uint256 tAVAXReceived = _wrapperDeposit(assets);

+    shares = _previewWrap(tAVAXReceived, Math.Rounding.Floor);
    _mint(receiver, shares);
    emit Deposit(caller, receiver, assets, shares);
}

/*////////////////////////////////////////////////////////////////
                    INTERNALS
////////////////////////////////////////////////////////////////*/

/// @dev Internal function to mint tAVAX shares by first depositing in the sAVAX vault.
- function _wrapperDeposit(uint256 amount) internal {
+ function _wrapperDeposit(uint256 amount) internal returns (uint256 tAVAXReceived) {
    if (amount != 0) {
+        uint256 tAVAXBefore = IERC20(vaultShare()).balanceOf(address(this));
        // Treehouse router handles the deposit of wAVAX into sAVAX and then into tAVAX.
Returns tAVAX to this contract.
        ITreehouseRouter(treehouseRouter).deposit(wAVAX, amount);
+        tAVAXReceived =  IERC20(vaultShare()).balanceOf(address(this)) - tAVAXBefore;
    }
}
```

Status

✓ Fixed

[WP-G3] Using `immutable` can save gas

Gas

Issue Description

The variables `wAVAX`, `sAVAX`, and `treehouseRouter` are never modified after initialization. They should be marked as `immutable` to avoid unnecessary SLOAD gas costs.

https://github.com/treehouse-gaia/spectra-wrapped-tavax/blob/a4526cab672ad5ea42ca7174603f8a27fad0c1ff/src/Treehouse/SpectraWrappedtAVAX.sol

```
@@ 1,10 @@
11
@@ 12,15 @@
16  contract SpectraWrappedtAVAX is Spectra4626Wrapper {
17      using Math for uint256;
18      using SafeERC20 for IERC20;
19
20      address public  wAVAX;
21      address public  sAVAX;
22      address public  treehouseRouter;
23
24      error WithdrawNotImplemented();
25      error RedeemNotImplemented();
26
27      constructor() {
28          _disableInitializers();
29      }
30
31      function initialize(
32          address _wavax,
33          address _savax,
34          address _tavax,
35          address _treehouseRouter,
36          address _initAuth
37      ) external initializer {
38          __Spectra4626Wrapper_init(_wavax, _tavax, _initAuth);
39          IERC20(_wavax).forceApprove(_treehouseRouter, type(uint256).max);
40          wAVAX = _wavax;
41          sAVAX = _savax;
```

```
42          treehouseRouter = _treehouseRouter;
43      }
44
        @@ 45,151 @@
152     }
```

Status

✓ Fixed

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.